Messaging: block audit and two race fixes

Work report for the **messaging** worktree, 1 and 2 October 2026. Branch `feat/messaging`, commit `1ecc4acb`.

1. Summary

- The worktree block for the chat module was **already fully implemented**. An audit confirmed every task item and every acceptance scenario, with the 21 existing tests passing.
- With approval, a short closing plan was carried out: one documentation correction and **two defect fixes** that the load baseline of 29 September had found.
- **Racing posts are no longer lost.** A post refused because another post changed the conversation first is now dispatched again instead of failing.
- **A socket can attach right after a conversation is opened.** The handshake now waits briefly for the read model instead of refusing.
- The messaging suite grew from 21 to **32 tests, all passing**. The work is committed and not yet pushed or merged.
- **Not done:** the lost-post rate has not been measured again under the baseline load. Section 8 explains why.

2. Where the work started

The assignment was the block *Worktree: messaging* in `CLAUDE.md`. Before planning anything, the existing code on the branch was checked against each line of that block.

Item of the block	What the audit found
Aggregate `Conversation` with its commands and events	Present, covered by 10 aggregate tests
Schema moved to `announce_messaging.sql`, projections write it	Present and wired into the build script
WebSocket delivery, fan-out over `messaging.live`, REST fallback	Present, covered end to end
Public events on `messaging.v1`, subscriber on `order.v1`	Present; `OfferAcceptedV1` carries the agreed field names
Support queue and bulletins	Present, covered end to end
Acceptance: `mvn -pl messaging test`	21 tests green on Testcontainers plus the live MySQL
Acceptance: section 9 of `doc/order-context.md` records deviations	Two bullets were out of date

The branch was one commit ahead of `announce-jo-branch` and none behind. Two known defects from the baseline run were still open. They were not acceptance items, so they were proposed as optional steps.

3. The approved plan

1. Bring section 9 of `doc/order-context.md` up to date.
2. Retry posts that lose a race on one conversation.
3. Close the gap that refused a socket opened right after the conversation.
4. Verify with tests and record the result in `doc/messaging-architecture.md`.

All four steps were approved and done. No scope was added beyond them, apart from two small items that belong to step 2: a counter for the retries and a proper error code on the socket.

4. Fix one: posts at the same moment

The defect. When buyer and seller post at the same moment, both commands read the same state of the conversation. The event store accepts the first write and refuses the second. Nothing retried, so the second post failed and the socket answered error 500. Under the baseline load 12 of 800 posts were lost this way.

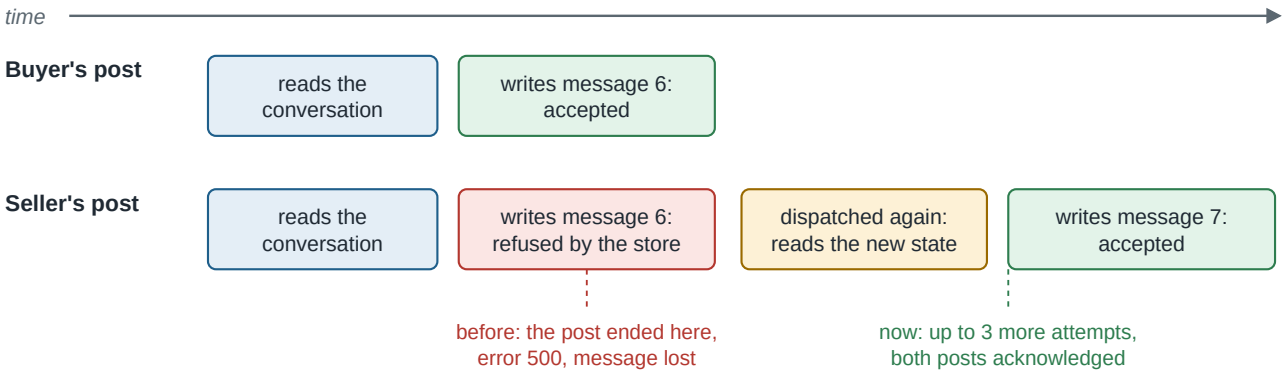

Figure 1. Two posts handled together. The refused write stores nothing, so running the command again on the new state is safe.

The fix. The command facade `Conversations.send` now dispatches a refused command again, at most three times, with a short random pause between attempts. Each failed attempt means another writer got through, so up to four simultaneous writers always succeed.

- A new counter, `messaging.command.retries`, shows how often posts collide.
- A post that still loses after the last attempt answers 409 `concurrent-modification` on the socket, as REST already did. The client can send it again with the same `clientMsgId`.
- The retry is in the messaging module only. The shared command router is unchanged.

5. Fix two: a socket right after opening

The defect. The socket handshake checks the read model to decide whether the caller is a participant. The read model is written by a projection about a tenth of a second after the command answers. A client that connected immediately after opening a conversation was therefore closed with code 4403.

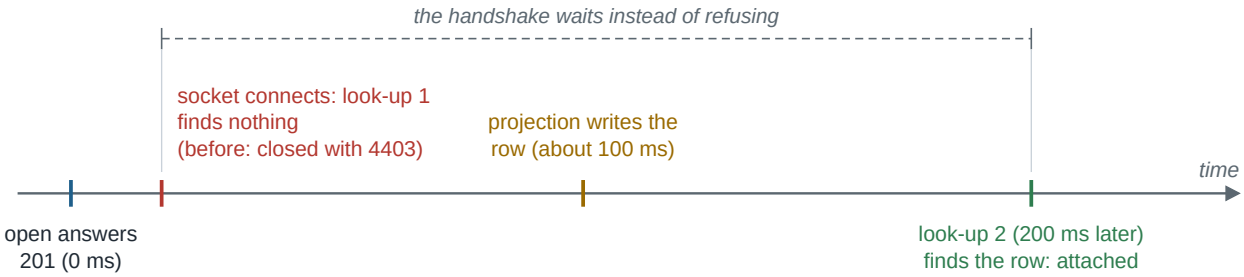

Figure 2. The handshake asks the read model before the projection has written the conversation. It now asks again.

The fix. `ConversationQueries.mayAttach` looks up a conversation that is missing from the read model up to five more times, 200 ms apart, before the socket is refused.

- Only a *missing* conversation is looked up again. A stranger who tries to attach to an existing conversation is still refused at once.
- An id that never existed costs about one second before the refusal.

Settings added by the two fixes

Setting	Default	Meaning
`messaging.command-retry.max-retries`	3	Further attempts after the first one; 0 turns the retry off
`messaging.command-retry.min-backoff`	5 ms	Pause before the first further attempt; doubles, with jitter
`messaging.command-retry.max-backoff`	50 ms	Upper bound of the pause
`messaging.attach.rechecks`	5	Further look-ups of a missing conversation; 0 refuses at once
`messaging.attach.recheck-interval`	200 ms	Pause between two look-ups

6. Documentation

Document	Change
`doc/order-context.md`, section 9	No longer says the `order.v1` records are local stubs: they come from `order-api`. States that the seller lookup is still an HTTP call. Records the two fixes as an addition.
`doc/messaging-architecture.md`	The two defects are described as fixed, with their limits. The new counter is in the meters table. Step 0 of the experiment order is marked done, with the load measurement named as still open.
`doc/messaging-context.md`	Explains the handshake look-ups, the retry, the 409 error frame and the new settings. Adds a warning about checkpoints after a test run (section 9 of this report).

7. Evidence

Test class	Before	After	What the new tests show
`ConversationTest`	10	10	
`ConversationControllerTest`	5	5	
`ConversationsRetryTest`		4	A refused post is dispatched again; the retry is bounded; a rejection is not retried; zero turns it off
`ConversationQueriesAttachTest`		5	A missing conversation is looked up again; the look-ups are bounded; a stranger is refused at once
`MessagingEndToEndTest`	6	8	A socket attaches right after opening. Ten rounds of buyer and seller posting at once, plus four simultaneous REST posts: 24 of 24 acknowledged
Total	**21**	**32**	

- **The tests catch the defects.** With both fixes switched off by configuration, the two new end-to-end tests fail: the socket is refused, and the first racing round already loses a post.

- **The retries really happen.** One recorded run counted 15 re-dispatches and no lost post. The suite asserts that the counter is above zero and that no post ended in error.
- **Final run:** 32 tests, no failures, on Testcontainers for the event store and Kafka and the live MySQL for the read model.

8. Limits and open items

- **Baseline load not rerun.** The stated measure for the retry is a zero error count under the load of 29 September. Starting a node for that run would have replayed about 2,500 old conversation events onto the public topics (see section 9), so it was left for a deliberate load run.
- **The retry is bounded.** A writer facing a continuous stream of posts from another socket can still use up its attempts and receive the 409.
- **The handshake waits about one second at most.** The slowest projection delay in the baseline was 770 ms. A slower read model would still refuse the socket; the two settings can be raised.
- **Other services do not get the retry.** Moving it into the shared command router would cover the order and announce modules and is a framework follow-up.
- **Seller lookup is still HTTP.** A local snapshot from `announce.v1` would remove the call to announce-management.

9. Two hazards met during the work

Concurrent test runs collide. While the suite ran here, a session in the `gateway-api` worktree started the same messaging tests against the shared live database. Its start-up cleanup deleted this run's test rows, and a bulletin test that was not touched failed once. The rerun in a quiet window was fully green. Only one worktree should run `MessagingEndToEndTest` at a time.

Checkpoints after a test run are wrong for a local node. The end-to-end test stores the progress of the three projection groups in the live database, as event ids of its own temporary event store. A node started afterwards against the local event store would resume from an id that means something else there. It would replay old events onto `messaging.v1` and `messaging.live`. Before starting a node, set the three rows in `cqrs_readside_progress` to the latest conversation event id, or delete them for a full rebuild. This is now written in `doc/messaging-context.md`, section 7.

10. What is in the commit

Commit `1ecc4acb`, *Retry racing posts and let a socket attach right after opening a conversation*, 2 October 2026. Thirteen files, 542 lines added and 40 removed.

File	Change
`application/Conversations.java`	Bounded retry on a refused write; counts the retries
`application/ConversationQueries.java`	The handshake check looks a missing conversation up again
`application/MessagingProperties.java`	Settings for the retry and the look-ups
`infrastructure/metrics/MessagingMeters.java`	New counter `messaging.command.retries`
`interfaces/ws/ConversationWebSocketHandler.java`	A lost race answers 409 instead of 500
`resources/config/application.yml`	Defaults of the new settings
`ConversationsRetryTest.java` (new)	Four unit tests of the retry

File	Change
`ConversationQueriesAttachTest.java` (new)	Five unit tests of the look-ups
`MessagingEndToEndTest.java`	Two new end-to-end cases and two meter assertions
`load/ChatLoadDriver.java`	Comment corrected; behaviour unchanged
`doc/order-context.md`, `doc/messaging-architecture.md`, `doc/messaging-context.md`	See section 6

11. State and next steps

- **Branch:** `feat/messaging` is two commits ahead of `announce-jo-branch` and none behind. Nothing has been pushed or merged.
- **Next decision:** merge the branch into `announce-jo-branch`.
- **Next measurement:** reset the checkpoints, start one node and rerun the load driver to confirm a zero error count under the baseline load.
- **Next tuning step:** the state cache on the command side, step 2 of the experiment order in `doc/messaging-architecture.md`.